1)

J'ai commencé par forker le projet, puis je l'ai cloné en local. J'ai ensuite installé les dépendances avec la commande npm install. En lançant le projet avec npm run dev, j'ai obtenu l'erreur ERR_OSSL_EVP_UNSUPPORTED. Elle venait d'une incompatibilité entre Next.js 11, , et Node.js 24. Next.js 11 utilise l'algorithme de chiffrement (MD4) que les versions récentes de Node.js bloquent . Le problème a été régler en ajoutant l'option --openssl-legacy-provider aux scripts dev, build et start du fichier package.json.

2)

J'ai regardé les différentes données affichées par l'application (MainCard, MetricsBox, DateAndTime) afin de construire, à l'aide de la documentation d'Open-Meteo, un appel API qui y corresponde. J'ai ensuite testé l'appel sur mon navigateur pour voir sous quelle forme l'API renvoie les données.

3)

J'ai créé un dossier configjson contenant le fichier weather.json, dans lequel est renseigné la ville de l'écran : ses coordonnées (latitude et longitude) pour l'appel à l'API, ainsi que son nom et son pays pour l'affichage.

4)
Le fichié data.js a été modifié pour utiliser les coordonnées dans le fichier weather.json (plus besoin d'utiliser une clef api) au lieu de recevoir une ville tapée par l'utilisateur. La fonction appel l'URL Open-Meteo avec ces coordonnées, récupère la réponse, y ajoute le nom de la ville et le pays, puis renvoie le tout à l'interface.

5)
Le fichier weatherCodes.js sert à associer chaque les code météo WMO (doc open-meteo) à une description et à une icône déjà présente dans le projet (icons). La fonction getWeatherInfo affiche les infos.

6)

J'ai remplacé les noms de champs d'OpenWeatherMap (main.temp, sys.sunrise…), pour les adapter à ceux d'Open-Meteo.

- •index.js : le nom de la ville et le pays viennent maintenant du fichier de configuration. La description et l'icône sont obtenues avec ma fonction getWeatherInfo, à partir du code météo et de is_day. J'ai aussi remplacé le test d'erreur message par error, le champ qu'utilise Open-Meteo.
- •MainCard.js: la température et le ressenti utilisent current.temperature_2m et current.apparent_temperature.
- •MetricsBox.js: l'humidité, le vent, la direction du vent et la visibilité utilisent les nouveaux noms de champs. Le lever et le coucher du soleil utilisent daily.sunrise[0] et daily.sunset[0].
- •helpers.js: OpenWeatherMap donnait les heures en timestamp Unix, à convertir avec le fuseau horaire. Open-Meteo les donne directement en heure locale, au

format ISO ("2026-10-20T08:18"). J'ai donc simplifié getTime et getAMPM pour qu'elles gardent seulement la partie après le « T », et getWeekDay calcule le jour à partir de la date du jour.

•DateAndTime.js: je l'ai adapté au nouveau format de getTime et getAMPM, en utilisant current.time.

7)

Un setInterval a été ajouté dans le useEffect afin de rappeler la fonction de récupération des données toutes les heures. Un clearInterval, placé dans la fonction de nettoyage, évite que plusieurs minuteurs tournent en même temps. Les données étant stockées dans un état React, l'affichage se met à jour automatiquement à chaque rafraîchissement.

8)
Pour finir, j'ai fait un nettoyage pour retirer tout ce qui servait à la recherche de ville, puisque la ville vient maintenant du fichier de configuration. Dans index.js, j'ai supprimé la barre de recherche (dans l'en-tête et sur l'écran d'erreur), les états qui stockaient la ville tapée, et l'appel en POST, remplacé par un simple GET. Le code retiré est en commentaire, sous le titre « Nettoyage recherche par texte ». J'ai ensuite supprimé les fichiers qui ne servaient plus : le composant Search et son CSS, ainsi que .env.example, qui contenait la clé de l'ancienne API. Enfin, j'ai mis à jour le README : description du projet, fonctionnalités, instructions d'installation sans clé API, et liste des fichiers supprimés.

Amélioration possible

Le multilingue (i18n) : les textes sont en anglais alors que les écrans sont destinés à des usagers en France. J'ajouterais un champ "language" dans weather.json et des fichiers de traduction (fr.json, en.json), gérés avec une bibliothèque comme next-i18next. Je devrais apprendre à utiliser une bibliothèque d'internationalisation.

Les tests unitaires : Lors de ma formation DWWM j’ai eu une initiation aux tests unitaires des tests unitaires . Pour ce projet je choisirai Jest pour tester les fonctions de services/, par exemple vérifier que getWeatherInfo renvoie « Clear sky ». Je devrais surtout apprendre la syntaxe de Jest

La mise à jour de Next.js : la version 11 n'est pas compatible avec Node.js récent, d'où le script dans le package.json --openssl-legacy-provider. Je mettrais Next.js àen m'appuyant sur les tests pour vérifier que rien n'est cassé. Je devrais apprendre à gérer une migration de version.

L'accessibilité : les écrands doivent être lisibles par tous, y compris les personnes malvoyantes, et de loin. J'agrandirais les textes et renforcerais les contrastes. Je chercherais à regarder si les textes alternatifs sont suffisamment précis **.** Je devrais apprendre les règles du RGAA et utiliser des outils d'audit comme Lighthouse.